

CSEN 175

Lab 1: Phase 1

TA Office Hours

Hudson Strauss

Email: hstrauss@scu.edu

Office Hours: T/Th 10am-12pm (Heafey Atrium)

Tiffany Nguyen

Email: tnguyen26@scu.edu

Office Hours: Wed 5:15-6:15 Th 1pm-2pm

Scott Wang

Email: hwang11@scu.edu

Welcome to Compilers

Motivation:

- Understand compiler technologies
- Be able to deal with a very complex project across several files
- Gain experience with C++ programming

To Keep in Mind:

- Time intensive
- Detail intensive
- Dr. Atkinson will absolutely hold you back from graduation
 - Some people have had to take compilers 3+ times
- The lab is a separate grade, capped at your final lecture grade
- 100% doable if you fully exercise your cognition and resources :)
- You get a clean, correct phase from Dr. Atkinson each time

DO NOT CHEAT

(discussion on this)

How to Ask for Asynchronous Help

Do:

- Email Dr. Atkinson, Tiffany, Scott, and Hudson all in the same email
- Include a *clean* (make clean) copy of your phase N .tar
- Include a detailed description of what's wrong
- Include a detailed description of what you've done to try to solve the problem

Don't:

- Email Dr. Atkinson, Tiffany, Scott, or Hudson individually
- Send screenshots or individual files
- Just say "I have no idea what's going on" and dump a ton of code

The “don’t”s are a sure-fire way to shoot your email to the bottom of the priority list!

Review of what the TA *is* and *is not* here for!

There are 3 kinds of questions you can ask:

1. I don't understand how to build a compiler
 - a. That's ok! So long as you've checked the lab slides, PDF, and lecture slides—and you're still confused as to how to structure your compiler—that's what we're here for! Ask away :)
2. I don't understand C++
 - a. These are problems you have to figure out on your own. You can Google each and every single one of these questions, C++ is a massive language with an even more massive amount of documentation on the web. You will get fired if you ask your employer how to use a string, where to find a function, or which method you should call on an object.
3. I don't understand how to program
 - a. This is a call for introspection if you truly should even be taking compilers. Everybody makes silly mistakes now and again, but consistently neglecting to run the code in your head, or *not visualizing how the compiler is navigating the data* as you write your program, is a recipe for a disastrous failure in this course.

Introduction

- Submissions

- Tar file uploaded to Camino
- Typically due on Sundays at 11:59PM (-1% for every minute late!)

- Advice

- Keep up with lecture material
 - *Or you will fail miserably and have to spend another \$20K for an additional quarter at SCU*
- Read the entire assignment (PDF & slides!) very carefully (most questions can be answered there)
- Start early
 - *A 2 week lab does not mean screw off for a week and a half then struggle at the last minute*
- Write your own test cases
 - Dr. Atkinson's provided ones do **NOT** cover all possibilities – you must fully exercise your own code!

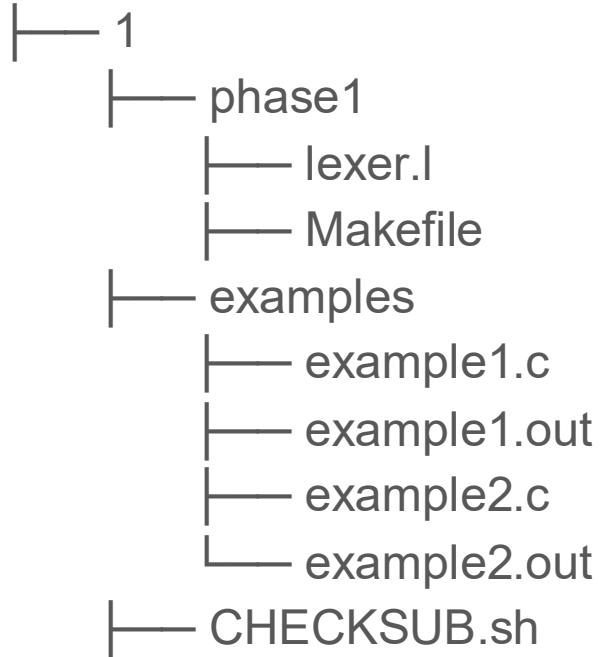
- Must run and compile on the ARM Compilers Research Server (ampere.engr.scu.edu)

- Make sure that you use the ORIGINAL makefile when doing so! – all of your own changes must be reverted prior to submission!

How Our Compiler Works

- Reads from standard input
- Writes to standard output
- Running
 - Using file redirection:
 - `./scc < example.c > output.out`
 - Using keyboard input:
 - `./scc`
 - Press Ctrl+D to signal EOF
- Testing
 - `diff output.out example.out`
 - Program is correct if there is no output

Recommended Directory Structure



Submitting

1. Create tar file (phase1.tar) containing your source code
 2. Run CHECKSUB to make sure tar file works
 - a. `./CHECKSUB.sh phase1.tar examples.tar`
 - i. **If you get a permission error at this point, run: `chmod 770 CHECKSUB.sh`**
 3. Submit tar file to Camino
- Run CHECKSUB before you leave lab today to make sure that you do not have issues running it later
 - **Feel free to submit early and often, we don't want you to have problems with submitting! It's okay to submit when partially complete.**
 - Always submit the **same exact** tar that you ran through CHECKSUB
 - **If CHECKSUB does not compile your submission, you will receive a 0.**

Phase 1 - Lexical Analysis

- Use flex to generate a lexical analyzer
- Print out every lexical construct (token) recognized from standard in
- Whitespace, comments, and illegal characters are ignored
 - All rules are on the assignment document
 - You will only be given lexically correct Simple C code in your tests for this phase
- Example
 - Standard in: `int x;`
 - Standard out: `keyword int, identifier x, operator ;`
- Due Sunday, September 27th at 11:59PM
 - ***-1% for every minute it is late***
 - **PLEASE RUN CHECKSUB ON WHAT YOU SUBMIT**

Anatomy of a flex File

A flex file has three sections, separated by a line containing %%

Definitions section

- C++ code and #includes go between %{ and %}
- Named patterns and %option declarations live here too

Rules section

- A list of pattern-action pairs
- the pattern is a regular expression; the action is C++ code in { }

User code section

- Optional extra C++ functions (often left empty)

Tracking Keywords

- Give each keyword its own rule (its own pattern-action pair)
 - int, return, while, ... — see the full list on the assignment doc
- Put the keyword rules BEFORE the identifier rule
 - flex uses the longest match; on a tie it uses the first matching rule
 - so “while” matches the keyword rule, not the identifier rule
- Print each keyword as: keyword <text>
- Do I have to list them all individually?
 - This class is extremely flexible with respect to implementation strategies. If your code is your own (don't cheat) and produces the right output, it will be accepted. Listing keywords as individual rules is the simplest approach, and what we recommend.

High Level View of the Lexer

- You mostly write regular-expression rules; flex generates the scanning loop for you
- Each rule's action prints what it recognized to `std::cout`
- Sample rules:
- `while` { `std::cout << "keyword while" << std::endl; }`
- `[a-zA-Z_][a-zA-Z0-9_]*` { `std::cout << "identifier " << yytext << std::endl; }`
- `[1-9][0-9]*` { `std::cout << "decimal " << yytext << std::endl; }`
 - `yytext` holds the exact text that matched the pattern

Writing Patterns with Regular Expressions

In flex you describe tokens with regular expressions, not `<cctype>` calls

Useful building blocks:

- `[a-zA-Z_]` — a letter or underscore
- `[0-9]` a digit, `[0-7]` an octal digit, `[0-9a-fA-F]` a hex digit
- `*` zero or more, `+` one or more, `?` optional

Escape characters that are special inside a pattern

- e.g. use `\"` for a double quote inside your string pattern

Remember: the longest match wins, and ties go to the earlier rule

Suggested Token Lexing Order:

1. Suggested order for your rules (earlier rules win ties):
2. Whitespace
3. Numbers (octal, decimal, hexadecimal)
4. Identifiers / keywords
 - a. Get at least up to here during the lab!
5. Operators and symbols
6. Strings / characters
 - a. escape the quote characters inside your patterns
7. Comments
 - a. Hard to match with a single regex — scan them by hand: on seeing `/*`, call `yyinput()` to consume characters until `*/` (use `unput()` to put one back)
 - b. Do not use the `nounput %option` if you call `unput()`
8. A final catch-all rule that reports any illegal character
 - a. CHECKSUB.sh only feeds you lexically legal programs, so this should never fire during grading.

Tips

- Make sure to spell keywords correctly!
 - This mistake happens every quarter, and it's a ridiculous way to lose points
- Test incrementally! Every time you add a new set of rules, test it at the command-line, as done by the example in the following slide.
 - This is not the kind of class where you can just dump all the lab's code into a file then try debugging the entire lab at once: especially as the phases progress, incremental testing will be the only way to reliably (and sanely) fix bugs.
 - You are given a Makefile that runs flex on your "lexer.l" file and compiles it into a "scc" binary
- Before **SUBMITTING**, MAKE SURE YOU PASS CHECKSUB.sh
 - Only do this after you have entirely completed the lab! CHECKSUB will not work on partially-completed labs.
 - Important that the file **YOU** TAR is the file that passes CHECKSUB **AND** is submitted!

Examples

```
[hstrauss@ampere:~/phase1$ make
c++ -std=c++17 -g -Wall -c -o lexer.o lexer.cpp
c++ -std=c++17 -g -Wall -c -o parser.o parser.cpp
c++ -std=c++17 -o scc lexer.o parser.o
[hstrauss@ampere:~/phase1$ ./scc
1
decimal 1
[-2
operator -
decimal 2
[typedef
keyword typedef
[int z;
keyword int
identifier z
operator ;
[0xFF
hexadecimal 0xFF
```

```
[hstrauss@ampere:~/phase1$ make clean
rm -f scc core *.o
[hstrauss@ampere:~/phase1$ cd ..
[hstrauss@ampere:~$ tar -cvf phase1.tar phase1
phase1/
phase1/Makefile
phase1/lexer.l
phase1/lexer.cpp
phase1/lexer.h
phase1/parser.cpp
[hstrauss@ampere:~$ ./CHECKSUB.sh phase1.tar examples.tar
Checking environment ...
Checking submission ...
Extracting submission ...
Compiling project ...
c++ -std=c++17 -g -Wall -c -o lexer.o lexer.cpp
c++ -std=c++17 -g -Wall -c -o parser.o parser.cpp
c++ -std=c++17 -o scc lexer.o parser.o
Extracting examples ...
Running examples ...
fib.c ... ok
hello.c ... ok
list.c ... ok
malloc.c ... ok
sum.c ... ok
tricky.c ... ok
```

```
[hstrauss@ampere:~/phase1$ ./scc < ../examples/hello.c > test.out
[hstrauss@ampere:~/phase1$ diff test.out ../examples/hello.out
hstrauss@ampere:~/phase1$ █
```